

BÉNINFOOD

PLAN DIRECTEUR DE RÉALISATION

De la décision métier à la mise en production, avec une seule application Flutter multi-rôles et une API Laravel centrale

DÉCISION D'ARCHITECTURE MVP

Béninfood utilise une seule application Flutter pour les trois expériences Client, Vendeur et Livreur. Le backend Laravel reste l'unique source de vérité métier, financière et de sécurité. Le back-office web consomme la même API et les mêmes services métier.

Élément	Décision retenue V1.2
Architecture mobile	Une seule application Flutter, package unique, organisée par features : auth, client, vendor, driver, core, shared, router.
Backend	Laravel API-first, version /api/v1, logique métier et calculs sensibles côté serveur.
Rôles	Un utilisateur peut posséder un ou plusieurs rôles. L'application active un contexte : client, vendeur ou livreur.
Sécurité	RBAC + policies + contrôles serveur. Le rôle déclaré par le mobile n'est jamais considéré comme une preuve d'autorisation.
Païement	FedaPay ou Kkiapay via une abstraction PaymentGateway réversible.
Commission	Taux configurable par la porteuse, historisé par commande et immuable dans le snapshot financier.
Livraison	Livreur indépendant ou travailleur Béninfood, avec zones et tarifs configurables.
Administration	Back-office web pour la porteuse; aucun accès direct à la base de données.
Evolution V2	Split éventuel de l'application uniquement si les usages, la taille, l'organisation ou un besoin white-label le justifient.

0. Résumé exécutif

Le présent document transforme le cadrage Béninfood en feuille de route exécutable. Le changement majeur de cette V1.2 est la décision de remplacer les trois applications Flutter distinctes prévues initialement par une seule application mobile multi-rôles. Le backend, le modèle financier, l'API et le back-office restent centralisés.

Principe directeur

On développe d'abord le cœur métier Laravel et le modèle financier. La même API sert l'application Flutter multi-rôles et le back-office web. Les règles de paiement, de commission, d'annulation, de livraison et de sécurité ne sont jamais copiées dans Flutter.

Le projet reste piloté comme une marketplace transactionnelle : catalogue -> panier -> commande -> paiement -> répartition comptable -> préparation -> livraison -> confirmation -> règlement/reversement -> historique -> support.

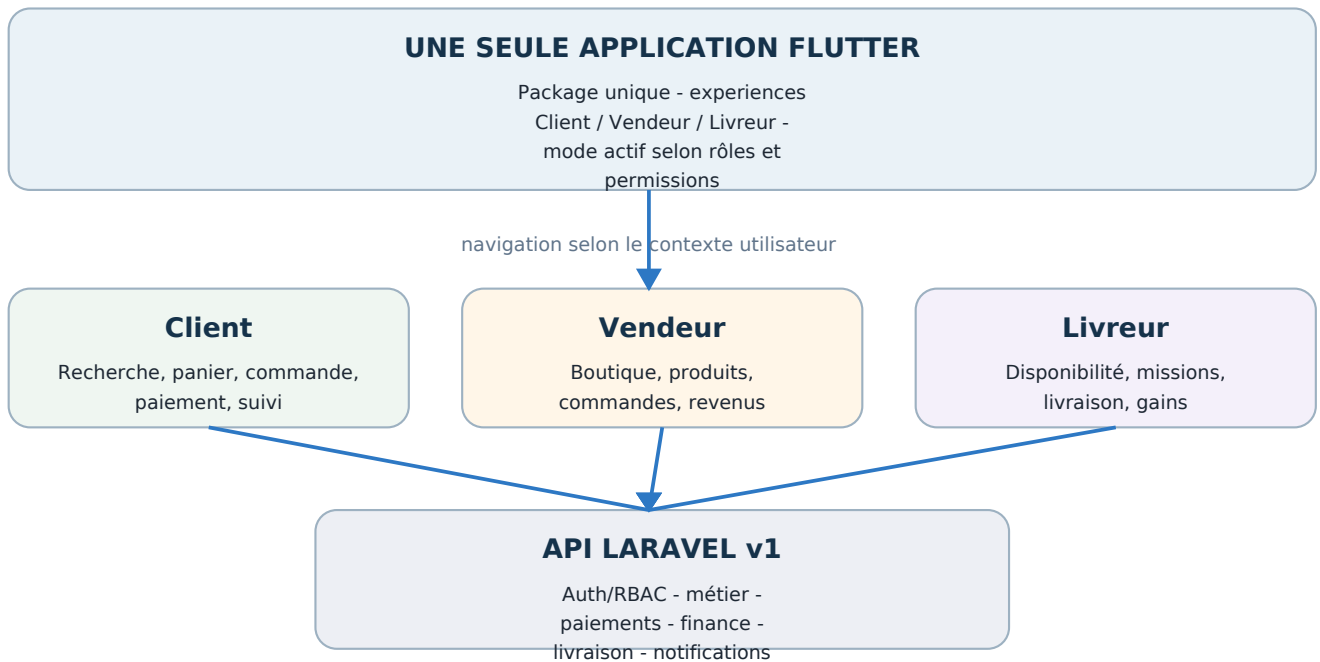
Résultat final attendu

- API Laravel documentée, sécurisée et testée.
- Back-office web complet pour la porteuse et les opérations Béninfood.
- Une application Flutter unique couvrant Client, Vendeur et Livreur.
- Paiement mobile money / carte via un agrégateur sélectionné.
- Commission configurable, historisée et financièrement traçable.
- Gestion des zones, tarifs, livraison, annulations, remboursements, réclamations et audits.
- Déploiement production, sauvegardes, monitoring et procédure de reprise.
- Documentation technique, guides et formation des utilisateurs.

0.1. Ce qui ne change pas

Le plan conserve les décisions métier de référence : API-first Laravel, centralisation des calculs financiers, back-office web, FedaPay/Kkiapay abstrait par PaymentGateway, livreurs indépendants ou Béninfood et tarification par zones. Le changement porte sur la distribution mobile, pas sur le métier.

1. Architecture cible



1.1. Organisation Flutter

Package unique : Béninfood Mobile. Les modules sont isolés par feature afin de permettre une future extraction sans réécriture du backend.

Dossier	Responsabilité
core/	Authentification, HTTP, stockage sécurisé, configuration, thème, gestion réseau.
shared/	Widgets et composants réutilisables entre les expériences.
features/auth/	Inscription, connexion, récupération, profil et gestion de session.
features/client/	Accueil, recherche, boutiques, produits, panier, checkout, paiement, suivi, historique, réclamations.
features/vendor/	Boutique, horaires, produits, disponibilité, commandes, préparation, revenus.
features/driver/	Disponibilité, missions, collecte, livraison, preuve, incidents, gains.
router/	Navigation et redirection selon l'utilisateur authentifié et son contexte actif.
data/domain/presentation/	Séparation recommandée entre données, règles de présentation et accès API lorsque nécessaire.

1.2. Principe multi-rôles

L'utilisateur authentifié obtient ses rôles depuis Laravel. Si plusieurs rôles sont disponibles, l'application propose une bascule explicite de mode. Cette bascule ne modifie pas les droits côté serveur; elle modifie uniquement le contexte d'expérience et la navigation.

Exemple : un utilisateur peut être à la fois client et vendeur. Il conserve un seul compte, une seule session et les mêmes mécanismes de sécurité, tout en passant d'un espace à l'autre.

1.3. Laravel comme API pour tout le système

Laravel fournit les endpoints versionnés, l'authentification, les politiques, les workflows, les calculs, les webhooks, les notifications, les journaux et les données. Flutter ne connaît pas les clés de paiement et ne calcule jamais seul les montants sensibles.

RÈGLE DE SÉCURITÉ

Aucune clé FedaPay/Kkiapay, Firebase ou autre secret ne doit se retrouver dans Flutter, Git ou les réponses API.

2. Modèle de données et contrats

Le modèle conserve les tables métier identifiées dans la version précédente, avec l'ajout explicite des structures nécessaires aux utilisateurs multi-rôles et aux appareils mobiles.

Bloc	Tables principales
Utilisateurs	users, roles, permissions, user_roles, user_devices, addresses
Vendeurs	vendors, vendor_documents, vendor_hours, vendor_settings
Catalogue	categories, products, product_images, product_variants, stocks
Commandes	carts, cart_items, orders, order_items, order_status_history
Finance	payments, payment_events, commission_rules, order_financials, wallets, wallet_transactions, payouts, refunds
Livraison	delivery_zones, delivery_rates, deliveries, delivery_status_history, driver_profiles, driver_documents
Support	reviews, complaints, complaint_messages, attachments
Notifications	notifications, notification_templates
Audit	audit_logs, webhook_events, system_logs

2.1. Snapshot financier à conserver

Pour chaque commande payée : sous-total, remise éventuelle, frais de livraison, frais de paiement éventuels, base de commission, taux appliqué, commission plateforme, montant vendeur, montant livreur, total client, devise, identifiant prestataire et références de transaction.

2.2. API versionnée

Toutes les routes publiques et mobiles sont versionnées en `/api/v1/`. Le back-office utilise la même couche de services et de permissions, sans logique métier parallèle.

Domaine	Endpoints essentiels
Auth	POST /auth/register, POST /auth/login, POST /auth/logout, GET /me
Rôles / contexte	GET /me/roles, POST /me/active-role
Vendeurs	GET /vendors, POST /vendors, GET/PUT /vendors/{id}, POST /vendors/{id}/approve
Catalogue	GET /categories, GET /products, POST/PUT/DELETE /products
Panier / checkout	GET /cart, POST /cart/items, PUT /cart/items/{id}, DELETE /cart/items/{id}, POST /checkout/quote, POST /orders
Commandes	GET /orders, GET /orders/{id}, POST /orders/{id}/accept reject prepare ready cancel
Paieement	POST /orders/{id}/payment, GET /payments/{id}, POST /webhooks/fedapay, POST /webhooks/kkiapay
Livraison	GET /deliveries, POST /deliveries/{id}/accept pickup deliver incident
Finance	GET /finance/summary, GET /vendors/{id}/earnings, GET /commissions, GET /payouts
Admin	GET /admin/dashboard, /admin/vendors, /admin/orders, /admin/payments, /admin/refunds, /admin/audit

3. Découpage en lots et ordre de réalisation

Lot	Nom	Dépendances	Sortie attendue
L00	Cadrage final	Aucune	Décisions signées + backlog
L01	UX/UI & parcours	L00	Maquettes validées pour les 3 contextes
L02	Fondation Laravel	L00	Projet API propre
L03	Auth & RBAC multi-rôles	L02	Profils et contexte actif sécurisés
L04	Vendeurs & boutiques	L03	Marketplace alimentée
L05	Catalogue & stock	L04	Produits publiables
L06	Adresses & zones	L04	Tarifs livraison
L07	Panier & checkout	L05,L06	Panier calculé côté serveur
L08	Commandes	L07	Workflow complet
L09	Paiement	L08	Paiement confirmé par webhook
L10	Finance & commission	L09	Ledger vendeur/plateforme
L11	Livraison	L08,L10	Dispatch + suivi
L12	Notifications	L08,L09,L11	Push opérationnelles
L13	Back-office	L03,L04,L08,L10,L11	Pilotage métier
L14	Flutter multi-rôles	API stable	Une app mobile complète
L15	Avis / réclamations / remboursements	L08,L09,L11	Support métier
L16	QA & sécurité	Tous	Recette prête
L17	Production	L16	Go-Live
L18	Formation & clôture	L17	Projet remis
L19	Maintenance V1	L18	Stabilisation + backlog V2

4. Règles d'architecture à verrouiller

- Laravel décide des permissions; Flutter ne fait qu'appliquer l'expérience correspondante.
- Les montants sont recalculés côté serveur avant paiement.
- Les webhooks sont vérifiés, dédoublés et traités de façon idempotente.
- Le contexte actif (client/vendeur/livreur) ne constitue jamais une autorisation en soi.
- Les notifications contiennent un type d'événement et un deep-link vers le bon espace.
- Les modules Flutter doivent être découplés afin de faciliter un éventuel split V2.

5. Phase 0 - cadrage final

- Créer le backlog officiel et identifier la porteuse comme Product Owner.
- Valider les rôles : porteuse/admin, vendeur, client, livreur indépendant, livreur Béninfood, super admin technique.
- Décider si un utilisateur peut cumuler plusieurs rôles et valider les règles de bascule de mode.
- Verrouiller l'agrégateur principal et obtenir les comptes sandbox.
- Valider commission, reversements, frais de paiement, annulations, remboursements et zones.
- Choisir hébergement, domaine et comptes Google Play / Apple.
- Valider les maquettes des expériences Client, Vendeur et Livreur dans une seule application.

6. Phases techniques détaillées

6.1. Fondation Laravel

- Initialiser dépôt et environnements local / staging / production.
- Installer Sanctum et permissions.
- Configurer migrations, factories, seeders, API Resources, Form Requests et exceptions.
- Configurer logs structurés, queues, jobs, scheduler, CI et documentation API.

6.2. Authentification et autorisation multi-rôles

- Inscription et connexion client.
- Inscription et validation vendeur.
- Inscription et validation livreur.
- Activation des livreurs Béninfood.
- Gestion du mot de passe, vérification de compte et sessions.
- Association user_roles et récupération des rôles via GET /me.
- Choix du rôle actif côté Flutter, avec contrôle serveur sur chaque endpoint.
- Tests d'accès interdit, accès horizontal, permissions et audit.

6.3. Coeur métier marketplace

- Vendeurs et boutiques.
- Catalogue, produits, images, disponibilité, stock.
- Zones et tarifs.
- Panier, quote, commande et state machine.
- Paiement, webhook, finance, commission et ledger.
- Livraison, affectation, prise en charge et preuve.
- Annulations, remboursements, réclamations et notifications.
- Back-office pour la porteuse.

6.4. Application Flutter multi-rôles

Contexte	Fonctionnalités principales
Commun	Onboarding, connexion, profil, stockage sécurisé, gestion session, réseau, erreurs, notifications, deep-links, changement de mode.
Client	Accueil, recherche, vendeurs, produit, panier, adresse, livraison, paiement, suivi, historique, avis, réclamation.
Vendeur	Onboarding, activation, boutique, horaires, produits, stock/disponibilité, commandes, préparation, historique, revenus.
Livreur	Inscription/activation, disponibilité, missions, acceptation/refus, collecte, livraison, preuve, incidents, historique, gains.

6.5. Politique de chargement mobile

Ne pas initialiser toutes les expériences au démarrage. Les features Client, Vendeur et Livreur sont chargées à la demande selon le contexte actif. Les services communs restent légers et partagés.

6.6. Géolocalisation et notifications

La localisation est utilisée selon consentement et besoin. FCM est commun à l'application unique, mais chaque notification porte son contexte et son deep-link. Les événements sont persistés côté serveur avant envoi asynchrone.

7. QA, sécurité et recette

Axe	Tests minimum
Fonctionnel	Parcours complets Client, Vendeur et Livreur dans la même application.
Paieement	Success, échec, attente, annulation, remboursement, doublon webhook.
Finance	Commission, frais, arrondis, changement de taux, remboursement partiel.
Commande	Stock 0, changement prix, vendeur fermé, annulation selon statut.
Livraison	Zone non desservie, absence livreur, refus, incident, changement de zone.
Sécurité	RBAC, accès horizontal, IDOR, rate limit, validation uploads, secrets.
Mobile	Connexion, changement de mode, perte réseau, reprise, push, deep-links, permissions.
Performance	API critiques, base, queue, pics d'événements webhook.
Sauvegarde	Backup + restauration sur staging.

7.1. Scénarios E2E obligatoires

- Client crée un compte -> choisit un vendeur -> panier -> zone -> paiement -> commande payée.
- Vendeur reçoit -> accepte -> prépare -> marque prête.
- Livreur indépendant accepte -> récupère -> livre -> commande delivered.
- Un même utilisateur passe du mode client au mode vendeur sans créer de second compte.
- Total client = part vendeur + commission plateforme + part livraison selon modèle.
- Vendeur annule avant préparation -> politique financière appliquée.
- Webhook reçu deux fois -> une seule validation de paiement.
- Webhook en retard -> la commande reste pending sans être marquée payée à tort.
- Restauration base -> application opérationnelle sur staging.

GO / NO-GO

Si un test financier ou de sécurité critique échoue, la mise en production est NO-GO même si les écrans paraissent terminés.

8. Préparation production et Go-Live

Élément	À faire avant Go-Live
Serveur	OS à jour, Nginx, PHP, DB, Redis si retenu, firewall, HTTPS.
Domaine	DNS, certificats SSL, sous-domaines API / admin.
Secrets	Variables de production dans une gestion sécurisée.
Base	Migrations, index, comptes DB minimaux, backup automatique.
Queue	Worker supervisé, retry policy, failed jobs.
Païement	Compte live, clés live, webhook HTTPS, tests contrôlés.
Firebase	Projet production, clés/certificats mobile.
Mobile	Un seul package Flutter signé, fiche store, politique de confidentialité et assets.
Monitoring	Logs, alertes, uptime, erreurs 5xx, queues, webhooks.
Support	Canal de support, procédure incident, rôles de garde.

8.1. Pipeline recommandé

Git -> CI -> tests -> staging -> recette -> tag release -> migration contrôlée -> déploiement API -> smoke tests -> activation progressive de l'unique application mobile -> monitoring renforcé.

8.2. Critères de Go-Live

- Cahier consolidé et maquettes validés.
- API documentée et tests critiques verts.
- Païement live et webhooks vérifiés.
- Commission, zones, tarifs et remboursements vérifiés.
- Backups restaurés avec succès en staging.
- Application Flutter unique signée et préparée pour les stores.
- Comptes administrateurs créés avec permissions minimales.
- Support opérationnel et plan de rollback prêts.
- Porteuse et vendeurs pilotes formés.

9. Planning directeur S1-S16

Semaine	Objectif principal	Jalon
S1	Cadrage + paiement + finance + livraison + multi-rôles	Décisions métier
S2	MCD/MLD + architecture + UX	Architecture validée
S3	Maquettes + auth + fondation API	Cadrage terminé
S4	Vendeurs + boutiques	Démo vendeur 1
S5	Catalogue + stock + zones	Démo vendeur 2
S6	Panier + checkout	Parcours client
S7	Commandes + state machine	Commande E2E sans paiement
S8	Paiement sandbox + webhooks	Démo mi-parcours
S9	Commission + ledger + remboursements	Finance fonctionnelle
S10	Back-office métier	Pilotage central
S11	Livraison + livreurs	Course E2E
S12	Notifications + réclamations	Ops complètes
S13	Flutter multi-rôles - Client + socle commun	Mobile intégré
S14	Flutter multi-rôles - Vendeur + Livreur + changement de mode	Mobile métier complet
S15	QA + sécurité + recette	Recette formelle
S16	Production + formation	GO LIVE

10. Comment travailler chaque jour

- Prendre une seule tâche métier à la fois.
- Créer une branche Git courte.
- Faire migration + modèle + service + endpoint + test.
- Ajouter l'autorisation et les logs avant de considérer la tâche terminée.
- Tester dans Postman/Swagger avant de brancher Flutter.
- Faire une démonstration de chaque workflow complet.
- Ne jamais coder en avance une intégration externe non validée.
- Clôturer chaque ticket avec une preuve : test, capture, endpoint ou build.
- Fusionner régulièrement vers develop/staging et éviter les grosses branches de plusieurs semaines.

11. Definition of Done

- Code écrit et relu.
- Migration / seed si nécessaire.
- Validation des données.
- Policy / permission.
- Logs utiles.
- Tests automatisés ou tests manuels documentés.
- Documentation API mise à jour.
- Événement / notification si nécessaire.
- Cas d'erreur traité.
- Démonstration fonctionnelle.
- Ticket mis à jour avec résultat.

12. Registre initial des risques

Risque	Impact	Réduction
Paieement non adapté au split marketplace	Très élevé	Choisir tôt; PaymentGateway abstrait; valider contrat + sandbox.
Webhook perdu / doublé	Très élevé	HTTPS, signature, idempotence, queue, reconciliation.
Règles de commission ambiguës	Très élevé	Snapshot taux/base dans chaque commande.
Livraison complexe	Élevé	Zones + affectation simple au départ, automatisation ensuite.
Application multi-rôles trop large	Élevé	Chargement par feature, Definition of Done et mesure analytics.
Accès admin excessifs	Élevé	RBAC + audit + comptes séparés.
Stores mobiles retardés	Moyen	Un seul compte/app store, assets et conformité préparés dès le cadrage.
Sauvegarde non restaurable	Très élevé	Tests de restauration réguliers.

13. Décision V2 - critères éventuels de split

Le split en plusieurs applications n'est pas une dette technique à résoudre immédiatement. Il devient un choix produit/organisation uniquement si les mesures de la V1 le justifient.

- Les analytics montrent que les vendeurs n'utilisent pratiquement jamais le mode client, ou inversement.
- La taille du binaire et la consommation des ressources deviennent problématiques sur les appareils ciblés.
- L'équipe devient suffisamment grande pour maintenir plusieurs applications et pipelines stores.
- Un besoin white-label ou partenaire impose une application dédiée.
- Les exigences commerciales, opérationnelles ou stores rendent un package unique moins pertinent.

PRINCIPE D'ÉVOLUTION

Même en cas de split V2, Laravel, les contrats API, les règles métier, le modèle financier et les permissions restent le socle commun. Le travail V1 doit donc isoler les features Flutter plutôt que les mélanger.

14. Critères de finalité du projet

- Un client réel peut commander et payer.
- Un vendeur réel peut recevoir, accepter, préparer et annuler selon les règles.
- Un livreur indépendant ou Béninfood peut réaliser une livraison.
- La porteuse suit commandes, chiffre d'affaires et commissions sans accès DB.
- Chaque montant est explicable par le journal financier.
- Une commande payée ne peut pas être validée deux fois.
- Un remboursement laisse une trace complète.
- Les zones et tarifs sont administrables sans modifier le code.
- Les trois expériences mobiles utilisent une seule application et la même API avec les mêmes contrôles de sécurité.
- Une sauvegarde testée permet de restaurer le système.
- La V1 est taggée et le backlog V2 est séparé de la production.

15. Fiche de validation du périmètre

Décision	Valeur retenue	Validé le / visa
Architecture mobile	1 application Flutter multi-rôles	
Backend	Laravel API /api/v1	
Rôles cumulables	Oui / Non	
Agrégateur principal	FedaPay / Kkiapay	
Taux commission initial		
Base de calcul commission		
Frais de paiement à la charge de		
Modèle reversement vendeur		
Zones / tarifs		
Type livreur autorisé	Indépendant + Béninfood	
Règles annulation vendeur		
Règles annulation porteuse		
Politique remboursement		
Date cible Go-Live		

16. Utilisation du document

Ce document devient le fil directeur du projet. Toute nouvelle exigence doit être ajoutée au backlog et reliée à un lot, une dépendance, un test et un critère d'acceptation. La référence mobile officielle est désormais une seule application Flutter multi-rôles.